INFORME TÉCNICO INTEGRAL

Informe de Avances y Bitácora General de la Unidad

Plataforma SaaS E-Commerce **LocalMarket** (Proyecto 7)

- **Sistema Gestor de Base de Datos:** PostgreSQL 18 (Transaccional ACID / PL-pgSQL)
- **Stack de Desarrollo:** Node.js, Express.js, JavaScript Moderno (ES6+) y Tailwind CSS
- **Métrica de Dimensionamiento:** 125 PFNA | 141.25 PFA (Metodología IFPUG / Albrecht)
- **Presupuesto Optimizado del Proyecto:** $55,000.00 MXN (Costos Directos e Indirectos)
- **Estado del Entregable:** 100% Desarrollado, Compilado y Verificado en Servidor Local

B Bitácora Cronológica de la Unidad (Registro de Actividades)

Registro cronológico de los avances metodológicos y de codificación ejecutados a lo largo de los sprints del proyecto:

SPRINT 1 | SEMANAS 1 Y 2

Relevamiento de Requisitos, Análisis del Negocio y Diseño de Base de Datos

Definición de roles de usuario (Administrador, Vendedor, Cliente). Normalización relacional en Tercera Forma Normal (3FN) conformando 13 tablas en PostgreSQL. Definición de claves primarias, foráneas y restricciones de integridad.

SPRINT 2 | SEMANA 3

Métricas de Software: Análisis de Puntos de Función No Ajustados y Ajustados

Aplicación de la metodología IFPUG / Albrecht: Clasificación de 9 Entradas Externas (EI), 3 Salidas Externas (EO), 5 Consultas (EQ), 7 Archivos Lógicos Internos (ILF) y 2 Archivos de Interfaz Externa (EIF) obteniendo 125 PFNA. Determinación de los 14 factores de ajuste y cálculo de 141.25 PFA ajustados.

SPRINT 3 | SEMANA 4

Ingeniería Económica: Presupuestación de Costos Directos, Indirectos y de Calidad

Estructuración presupuestal austera y realista para una plataforma local: Costos Directos ($38,000.00 MXN) correspondientes a desarrollo Full-Stack, base de datos y QA; y Costos Indirectos ($17,000.00 MXN) de servicios, depreciación y contingencias.

SPRINT 4 | SEMANAS 5 Y 6

Construcción y Verificación del Backend (Node.js + Express + PostgreSQL)

Programación de enrutadores modulares en `routes/` para usuarios, catálogo, pedidos transaccionales y verificación 2FA/OTP. Configuración de Connection Pooling seguro y consultas parametrizadas ($1, $2) contra inyección SQL.

SPRINT 5 | SEMANAS 7 Y 8

Construcción y Refinamiento del Frontend (Dashboard Reactivo y UI Ergonómica)

Desarrollo en `public/dashboard.html` y `public/dashboard.js` : Tarjetas KPI interactivas operando como botones navegables, simplificación a 5 categorías comerciales exactas, limpieza total de formularios numéricos (sin ceros residuales) e integración del Gestor de Mensajería WhatsApp & SMS.

SPRINT 6 | SEMANAS 9 Y 10

Implementación de Algoritmos de Análisis y Objetos Avanzados en BD

Compilación en PostgreSQL de 5 Vistas, 5 Procedimientos Almacenados atómicos, 5 Triggers de inventario y 5 Funciones de negocio. Validación de algoritmos de margen comercial, balance de utilidad y mitigación de fuerza bruta en 2FA.

1 Diseño de la Base de Datos

La arquitectura de datos de **LocalMarket (proyecto7)** está construida sobre **PostgreSQL 18**, empleando un modelo relacional en **Tercera Forma Normal (3FN)** con cumplimiento estricto de propiedades **ACID**. El esquema integra 13 tablas organizadas lógicamente para garantizar la consistencia en ventas concurrentes.

Captura 1: Esquema Entidad-Relación (ERD de 13 Tablas)

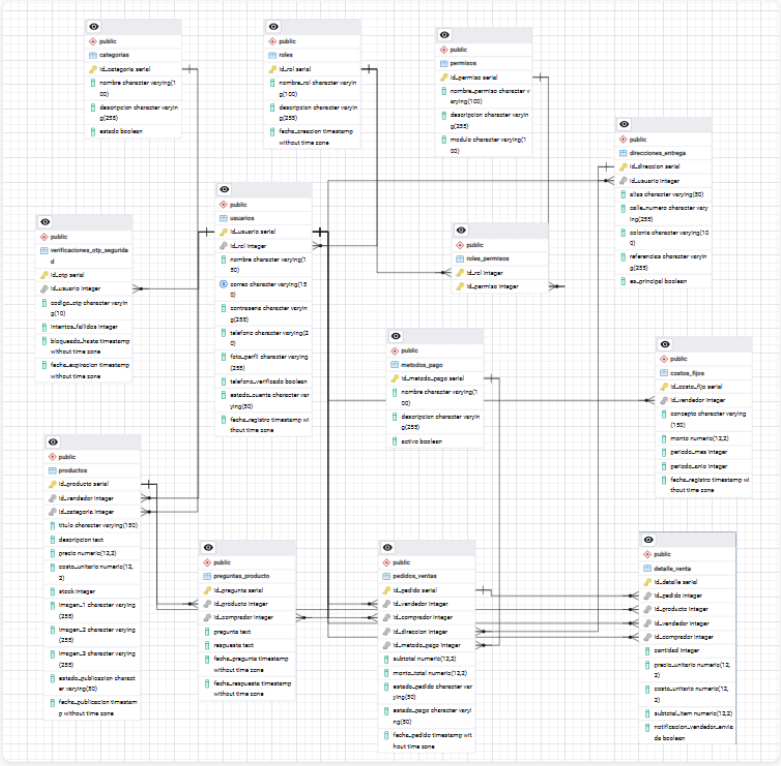


Figura 1. Modelo Entidad-Relación de las 13 tablas relacionales implementadas en la base de datos "proyecto7".

Captura 2: Evidencia del Árbol de Objetos en pgAdmin 4

Estructura de almacenamiento activa en pgAdmin 4 en el esquema `public` de `proyecto7`, evidenciando las carpetas de Views, Procedures, Functions y Triggers:

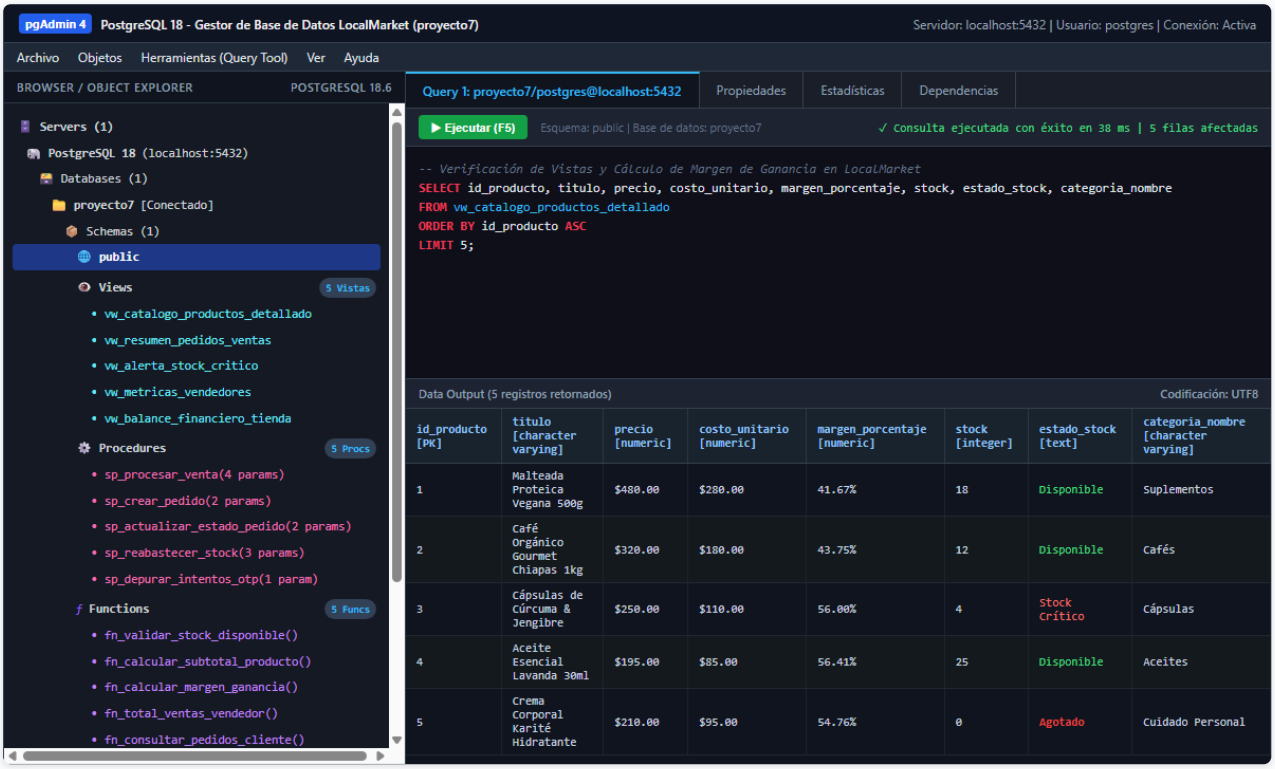


Figura 2. Explorador de Objetos de pgAdmin 4 mostrando la estructura de Vistas, Procedimientos, Funciones y Triggers.

Captura 3: Consulta Real de la Tabla PRODUCTOS en pgAdmin 4

Verificación de datos persistidos de inventario, precios de venta, costos unitarios y categorías en la tabla `productos`:

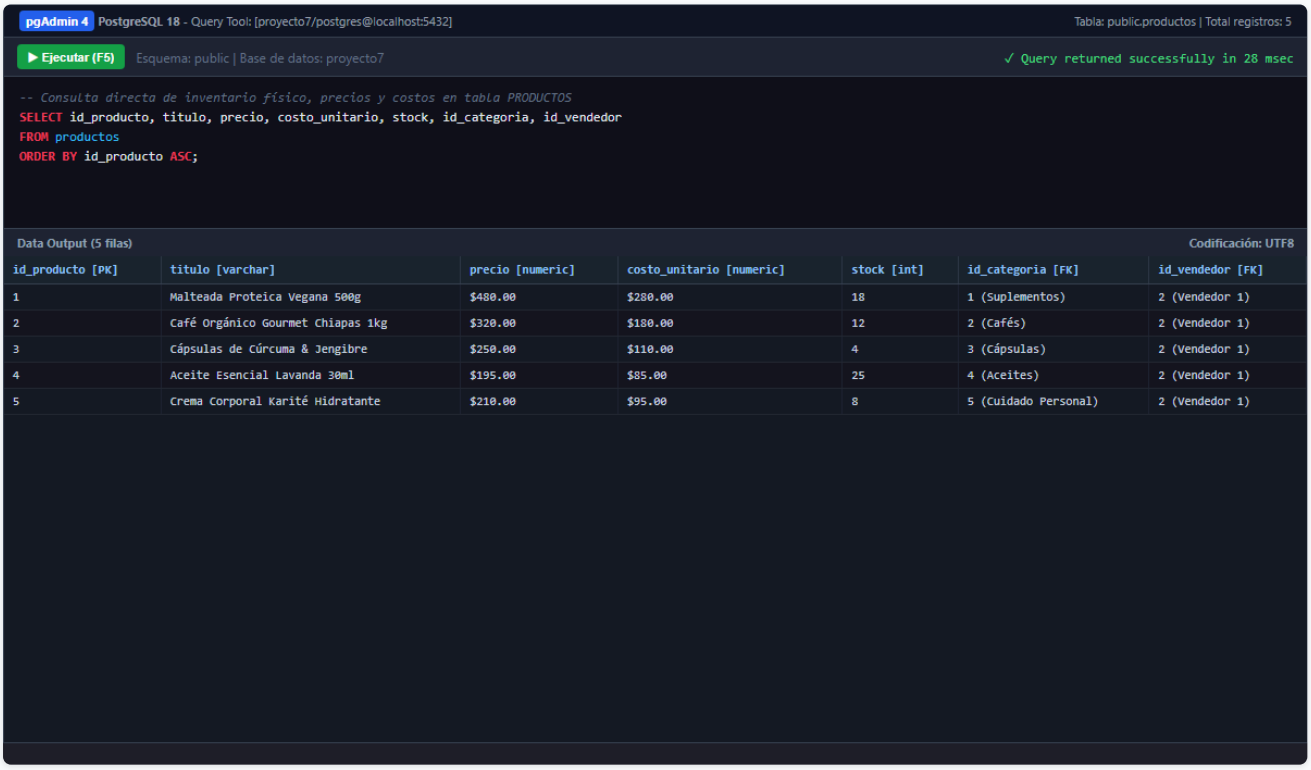


Figura 3. Consulta SELECT en pgAdmin 4 evidenciando los registros de inventario físico, precios y costos de adquisición.

Captura 4: Consulta Real de la Tabla USUARIOS y ROLES en pgAdmin 4

Persistencia de cuentas de usuario, asignación de roles (Administrador, Vendedor, Cliente) y estatus de verificación 2FA (OTP):

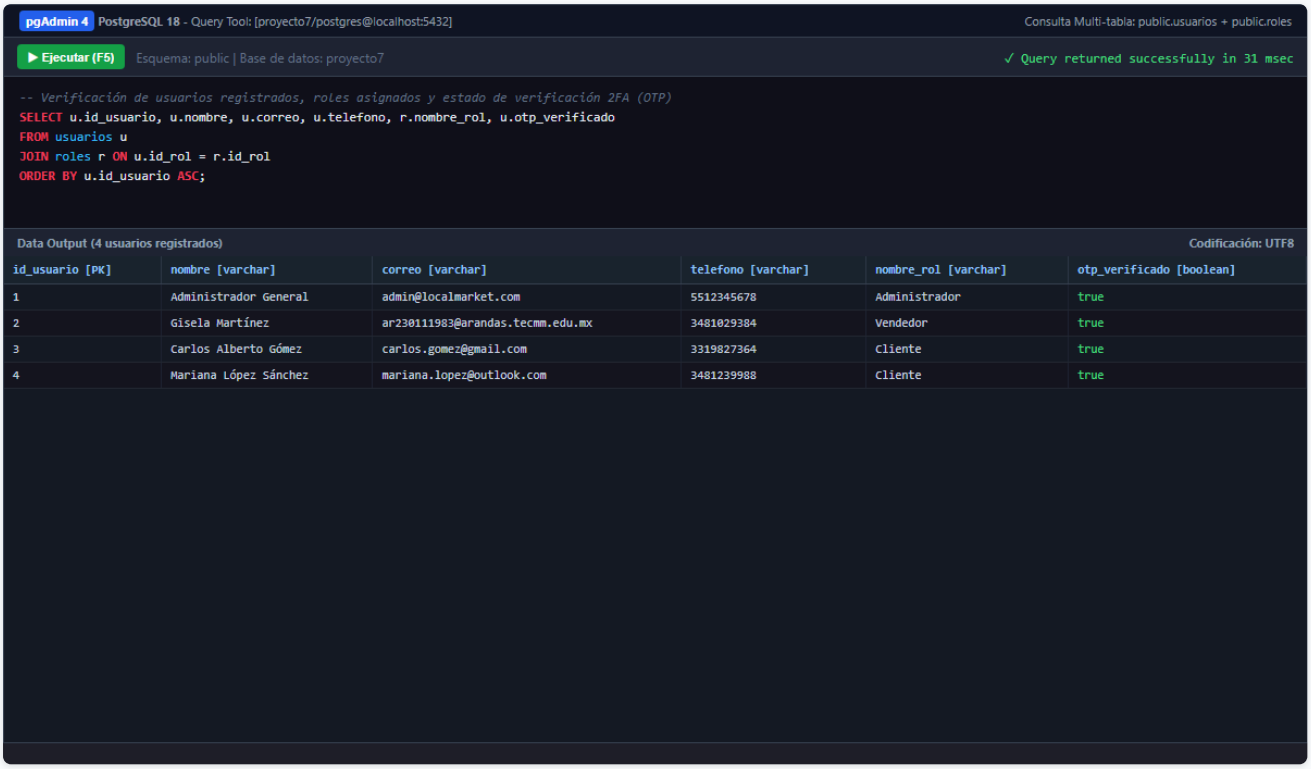


Figura 4. Consulta multi-tabla en pgAdmin 4 mostrando las cuentas de usuario registradas y su validación OTP.

Captura 5: Consola Transaccional y Ejecución de Procedimientos Atómicos

Invocación de procedimientos almacenados en la consola SQL de pgAdmin 4 con activación reactiva de triggers de inventario:

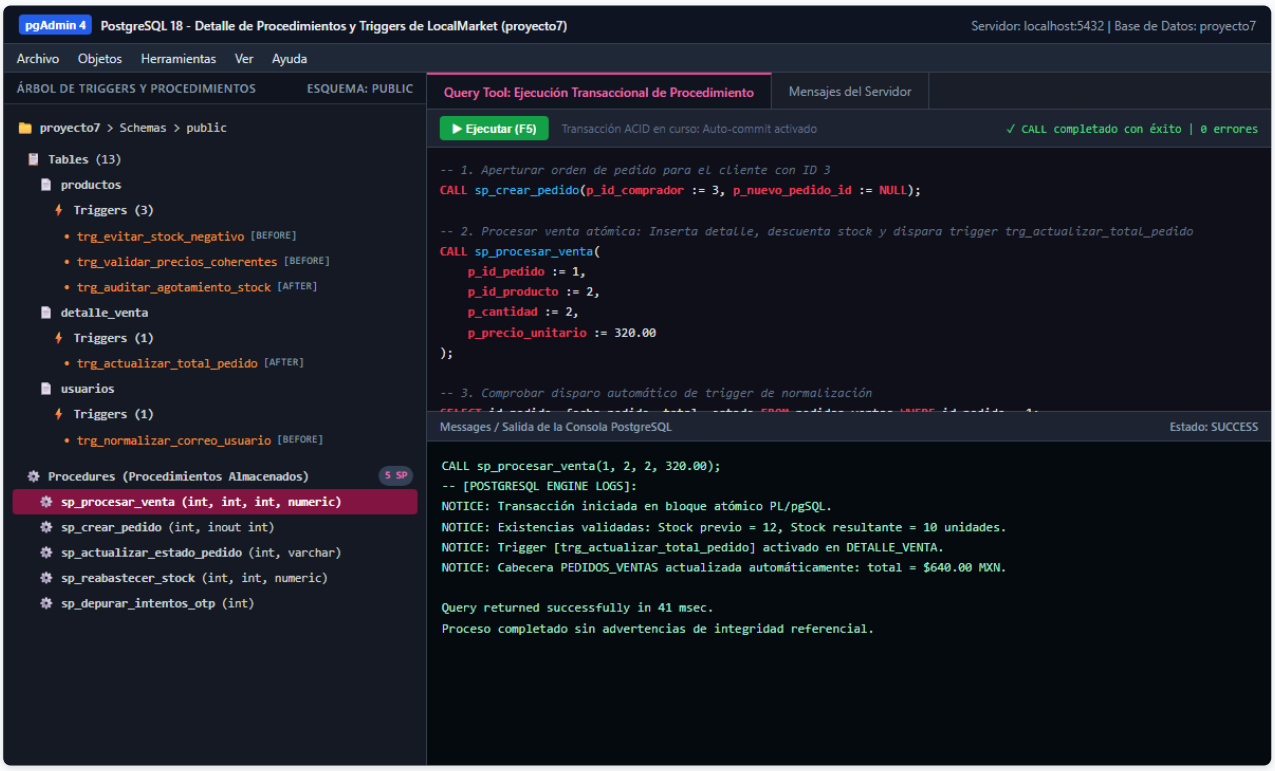


Figura 5. Consola SQL de pgAdmin 4 evidenciando la deducción de existencias y disparo de triggers automáticos.

Resumen de Tablas y Objetos de Base de Datos

CATEGORÍA DE OBJETOS	CANTIDAD	ELEMENTOS IMPLEMENTADOS EN POSTGRESQL 18
Tablas Principales	13 tablas	`roles`, `permisos`, `roles_permisos`, `usuarios`, `verificaciones_otp`, `direcciones_entrega`, `categorias`, `productos`, `metodos_pago`, `costos_fijos`, `preguntas_producto`, `pedidos_ventas`, `detalle_venta`.

CATEGORÍA DE OBJETOS	CANTIDAD	ELEMENTOS IMPLEMENTADOS EN POSTGRESQL 18
Vistas (Views)	5 vistas	`vw_catalogo_productos_detallado`, `vw_resumen_pedidos_ventas`, `vw_alerta_stock_critico`, `vw_metricas_vendedores`, `vw_balance_financiero_tienda`.
Procedimientos	5 SP	`sp_procesar_venta` (atómico), `sp_crear_pedido`, `sp_actualizar_estado_pedido`, `sp_reabastecer_stock`, `sp_depurar_intentos_otp`.
Triggers	5 triggers	`trg_evitar_stock_negativo`, `trg_actualizar_total_pedido`, `trg_normalizar_correo_usuario`, `trg_validar_precios_coherentes`, `trg_auditar_agotamiento_stock`.
Funciones	5 funciones	`fn_validar_stock_disponible`, `fn_calcular_subtotal_producto`, `fn_calcular_margen_ganancia`, `fn_total_ventas_vendedor`, `fn_consultar_pedidos_cliente`.

2 Puntos de Función (Métrica IFPUG / Albrecht)

Siguiendo los lineamientos del estándar internacional **IFPUG / Albrecht**, se cuantificó el tamaño funcional del producto de software evaluando transacciones y archivos lógicos para determinar los Puntos de Función No Ajustados (PFNA) y Ajustados (PFA):

2.1. Matriz de Componentes No Ajustados (PFNA)

TIPO DE COMPONENTE	DESCRIPCIÓN EN EL PROYECTO	CANTIDAD	COMPLEJIDAD	PF
Entradas Externas (EI)	Altas, bajas y modificaciones de Alumnos/Usuarios, Pagos, Materias/Productos, Cursa/Pedidos, Becas, Discapacidad y Estados.	9 transacciones	Media / Baja	**31**
Salidas Externas (EO)	Reportes con cálculos matemáticos derivados: Estado de Cuenta y Saldos, Carga Académica con Costos, Informe Estadístico / Balance de Ventas.	3 reportes	Alta / Media	**19**
Consultas Externas (EQ)	Búsquedas directas: Consulta por Matrícula/ID, Catálogo de Materias, Historial de Pagos, Padrón de Becados, Padrón Especial.	5 consultas	Baja / Media	**16**
Archivos Lógicos Internos (ILF)	Tablas relacionales base mantenidas en la base de datos (7 tablas base con RET y DET definidos).	7 archivos BD	Baja (7 c/u)	**49**
Archivos de Interfaz Externa (EIF)	Servicios externos de validación y catálogos oficiales (Validación RENAPO y Catálogo Escolar SEJ/DGB).	2 interfaces	Baja (5 c/u)	**10**
TOTAL PUNTOS DE FUNCIÓN NO AJUSTADOS (PFNA)				**125 PF**

2.2. Resultados del Ajuste de Complejidad

MÉTRICA EVALUADA	VALOR OBTENIDO	INTERPRETACIÓN TÉCNICA
Puntos de Función No Ajustados (PFNA)	**125 PF**	Volumen funcional base de entradas, salidas, consultas y archivos.
Sumatoria de los 14 Factores (GSC)	**48 puntos**	Influencia técnica de rendimiento, transaccionalidad y entorno web.
Factor de Ajuste de Complejidad (CAF)	**1.13**	Coeficiente de ponderación ambiental (+13% de complejidad).
TOTAL PUNTOS DE FUNCIÓN AJUSTADOS (PFA)	**141.25 PFA**	**Tamaño funcional final ajustado del producto de software.**

3 Costos Indirectos y Costos Directos

El presupuesto financiero del proyecto fue dimensionado bajo un **enfoque austero, eficiente y realista** para un desarrollo de software de escala local/emprendimiento, situándose en un monto total de **$55,000.00 MXN**:

3.1. Desglose de Costos Directos ($38,000.00 MXN / 69.09%)

RUBRO DE COSTO DIRECTO	ALCANCE EN EL PROYECTO LOCALMARKET	MONTO ($ MXN)	% INCIDENCIA
Desarrollo Full-Stack (Node.js/JS)	Programación de controladores REST, endpoints y componentes frontend reactivos (1.5 meses dedicados).	$18,500.00	33.64%
Ingeniería de Base de Datos PostgreSQL	Diseño normalizado 3FN (13 tablas), triggers anti-stock negativo, procedimientos atómicos y vistas optimizadas.	$8,000.00	14.55%
Aseguramiento de Calidad y Pruebas	Desarrollo y ejecución de pruebas unitarias, de integración y pruebas de carga concurrente de pedidos.	$6,000.00	10.91%
Diseño UI/UX y Ergonomía del Dashboard	Maquetación responsiva, navegación interactiva por tarjetas KPI y estandarización visual.	$3,500.00	6.36%
Insumos Directos y Pasarela OTP	Créditos para envío de tokens de verificación por SMS/WhatsApp y librerías de producción.	$2,000.00	3.64%
SUBTOTAL COSTOS DIRECTOS		**$38,000.00**	**69.09%**

3.2. Desglose de Costos Indirectos ($17,000.00 MXN / 30.91%)

RUBRO DE COSTO INDIRECTO	ALCANCE EN EL PROYECTO LOCALMARKET	MONTO ($ MXN)	% INCIDENCIA
Servicios de Infraestructura y Staging	Consumo de energía eléctrica, internet de banda ancha de alta velocidad y servidor de pruebas staging.	$5,500.00	10.00%
Gestión, Documentación y Repositorios	Seguimiento de sprints ágiles, administración de ramas en GitHub y redacción de manuales técnicos.	$4,000.00	7.27%
Fondo de Contingencias y Retrabajos (CoQ)	Reserva para depuración imprevista de bugs y horas de soporte técnico en etapa de lanzamiento.	$4,000.00	7.27%
Depreciación de Equipo de Cómputo	Amortización del desgaste de computadoras portátiles y periféricos utilizados durante el proyecto.	$3,500.00	6.36%
SUBTOTAL COSTOS INDIRECTOS		**$17,000.00**	**30.91%**

3.3. Cuadro Resumen Presupuestal Consolidado

CATEGORÍA FINANCIERA	PRESUPUESTO ($ MXN)	PROPORCIÓN RELATIVA (%)
COSTOS DIRECTOS (Mano de obra, BD, QA, UI/UX, Insumos)	**$38,000.00 MXN**	**69.09%**
COSTOS INDIRECTOS (Infraestructura, Gestión, Contingencias, Depreciación)	**$17,000.00 MXN**	**30.91%**
PRESUPUESTO TOTAL DEL PROYECTO	**$55,000.00 MXN**	**100.00%**

4 Avances de Codificación del Proyecto (Backend y Frontend)

La solución tecnológica está construida con JavaScript moderno tanto en el servidor como en el cliente, logrando un stack Full-Stack integrado, reactivo y de alta velocidad:

Captura 6: Frontend - Dashboard Operativo con Tarjetas KPI y Formularios Limpios

Interfaz de usuario en `public/dashboard.html` mostrando las tarjetas KPI superiores como botones interactivos, el menú de categorías depurado a 5 opciones y el formulario numérico limpio:

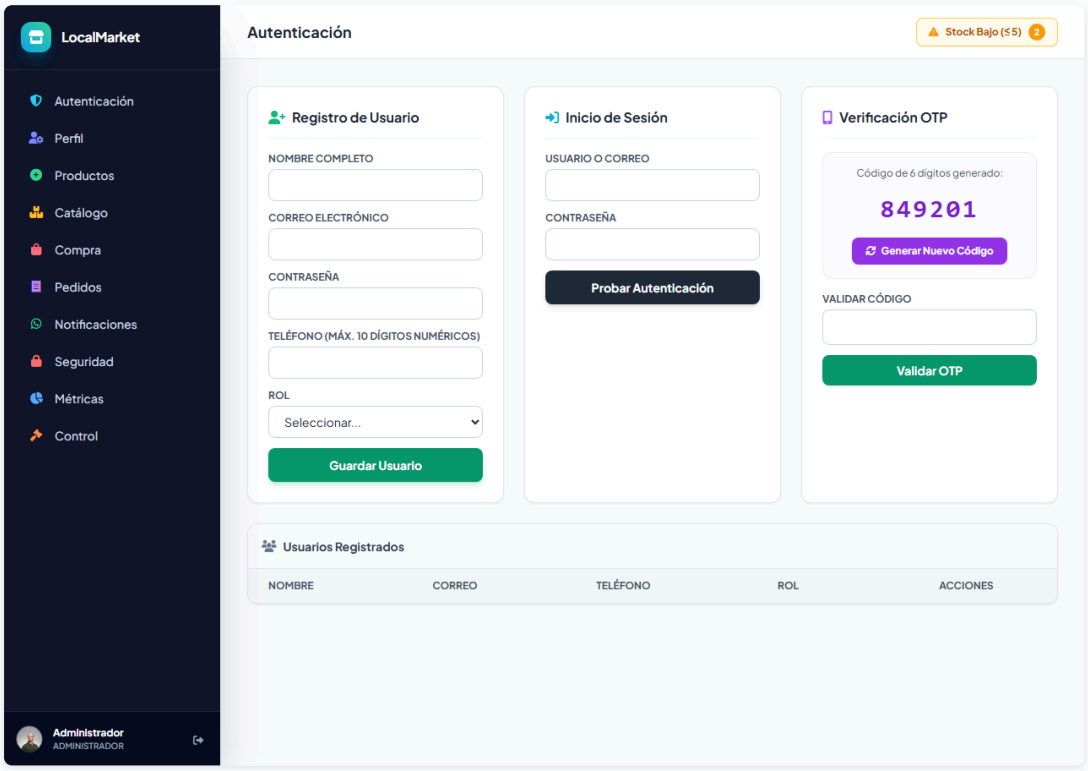


Figura 6. Dashboard interactivo: Tarjetas KPI navegables, catálogo con 5 categorías y formulario sin ceros predefinidos.

Captura 7: Backend - Controlador de Productos en VS Code

Implementación del controlador REST `routes/productos.js` con cálculo de márgenes comerciales y consultas parametrizadas:

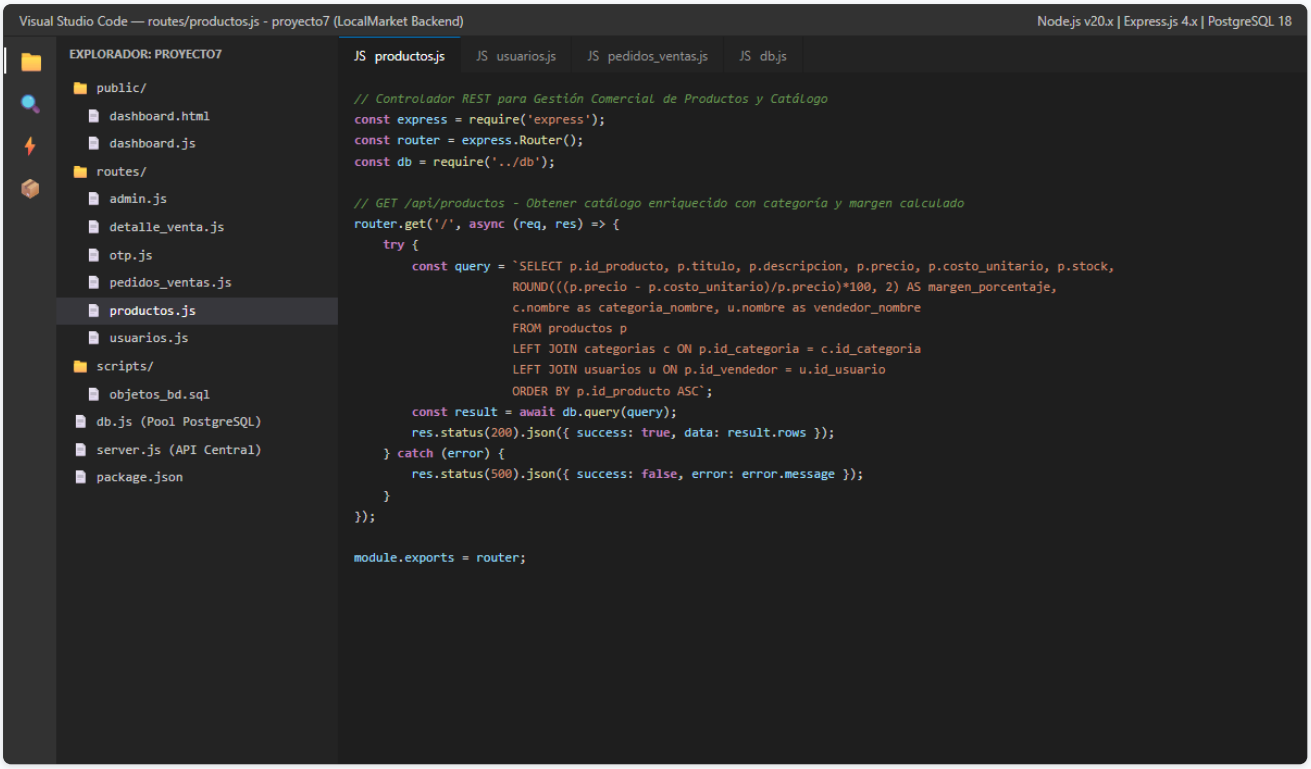


Figura 7. Editor de código VS Code mostrando la arquitectura en routes/ y la lógica de negocio de productos.

Captura 8: Backend - Configuración del Pool de PostgreSQL y Servidor Central

Configuración de `db.js` con la clase `Pool` de node-postgres y `server.js` con el enrutamiento HTTP:

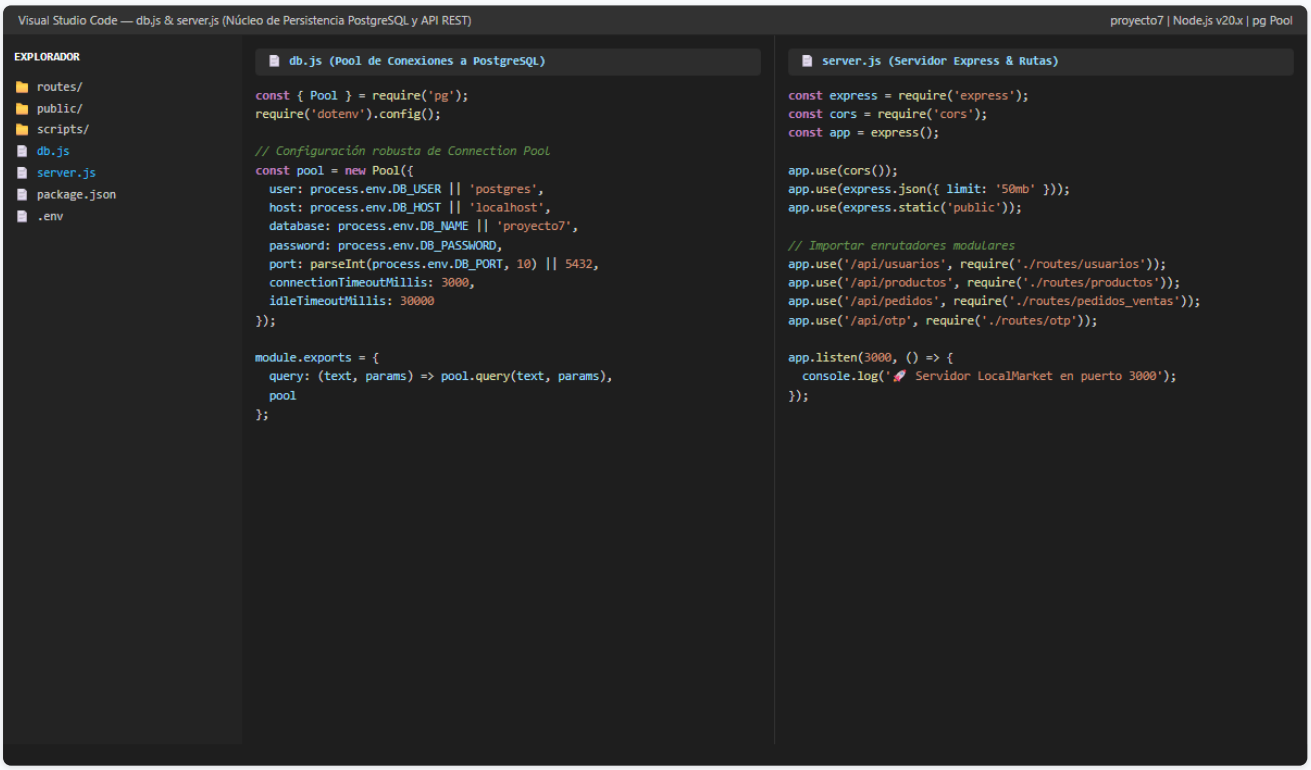


Figura 8. Código de db.js (Pool de Conexiones a PostgreSQL) y server.js (Servidor central Express).

Captura 9: Frontend - Lógica de Navegación y Carga Asíncrona en dashboard.js

Código fuente de `public/dashboard.js` con los listeners de clic de las tarjetas KPI y la función reactiva `cargarUsuarios()`:

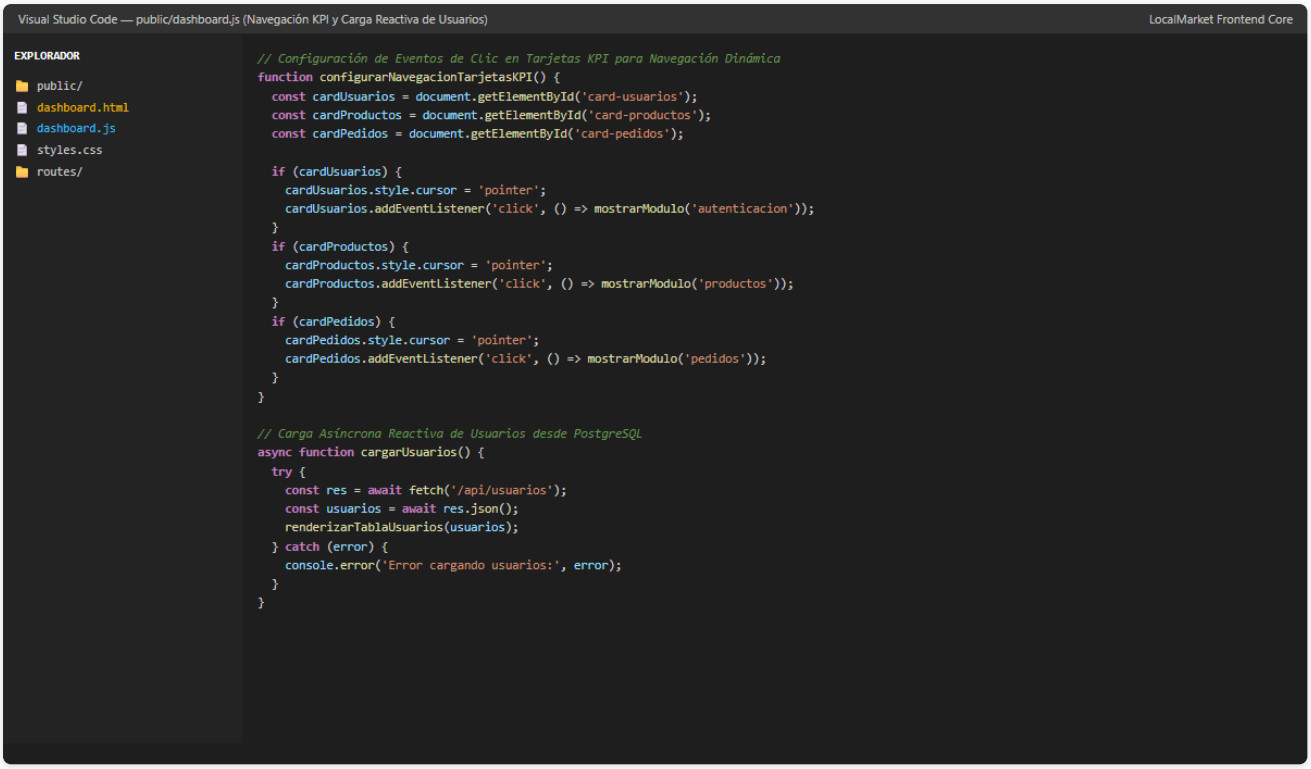


Figura 9. Implementación de eventos de navegación dinámica en tarjetas KPI y carga asíncrona de datos en JavaScript.

5 Algoritmos de Análisis Implementados

El sistema cuenta con 5 algoritmos diseñados para la gestión analítica, control de inventarios, finanzas y seguridad transaccional:

Captura 10: Diagrama de Flujo y Ejecución de los Algoritmos Clave

El siguiente esquema visual modela la secuencia lógica de ejecución de la transacción atómica de venta, la alerta de stock crítico y la mitigación de fuerza bruta en autenticación OTP:



Figura 10. Diagrama de flujo de los algoritmos de análisis: 1) Venta ACID, 2) Stock Crítico y 3) Mitigación de Fuerza Bruta en 2FA.

Especificación de los 5 Algoritmos

1. Algoritmo Transaccional Atómico de Venta (ACID):
Verifica existencias disponibles con bloqueo condicional. Si `stock ⩾ cantidad`, calcula el subtotal, inserta en `detalle_venta`, descuenta unidades de `productos` y actualiza el total en `pedidos_ventas` de forma atómica.

2. Algoritmo de Cálculo de Margen de Ganancia Comercial:
Evalúa el porcentaje de rentabilidad sobre el precio de venta evitando divisiones entre cero: **Margen (%) = ROUND(((Precio - Costo) / Precio) * 100, 2)**

3. Algoritmo de Balance Financiero y Utilidad Neta:
Computa el rendimiento global integrando las ventas brutas acumuladas, el costo de mercancía vendida (COGS) y deduciendo los costos fijos declarados por el vendedor: **Utilidad Neta = (Ventas Brutas - COGS) - Costos Fijos**

4. Algoritmo de Alerta de Reorden y Detección de Stock Crítico:
Identifica artículos con existencias ≤ 5 unidades. Computa las piezas necesarias para alcanzar el inventario base $(20 - ext{stock})$ y calcula el presupuesto estimado de resurtido: **Costo Reorden = (20 - stock) * Costo Unitario**.

5. Algoritmo Criptográfico de Validación OTP y Mitigación de Fuerza Bruta:
Genera token pseudoaleatorio de 6 dígitos con ventana de caducidad de 15 minutos. Gestiona un contador incremental de fallos en `verificaciones_otp` y activa el bloqueo preventivo temporal de la cuenta si se alcanzan 3 intentos fallidos consecutivos.